Loot Roll : Binary Exploit - Beginner

Creator: Claudia Varnas Points: –

Description

This is a challenge in which players will have to change the value of an int at runtime to gain access to a flag. They only have access to an executable file.

Github Path: Capture_the_Flame_2026/Binary_Exploit/Loot_Roll

Prompt

You and your party have just finished clearing through a difficult dungeon. It’s taken you days to get here, past the trials and tribulations and endless enemies, and now it’s time to claim your reward! A treasure chest gleams with the promise of loot. A miraculous helmet is inside, the final piece of gear you need. Hopefully the roll is in your favor.

Hints

No hints provided.

Solution

To solve this, I highly recommend using some sort of debugger that can modify memory at runtime. GDB works well for this in Linux/WSL. I have included an x64dbg solution as well. If you want to understand the assembly, please review the x64dbg solution, even if you won’t be using it.

DETAILED WRITEUP:

1. **(GDB). use “info functions”**

```
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x0000000000001000  _init
0x00000000000010b0  __cxa_finalize@plt
0x00000000000010c0  puts@plt
0x00000000000010d0  fclose@plt
0x00000000000010e0  printf@plt
0x00000000000010f0  fgets@plt
0x0000000000001100  malloc@plt
0x0000000000001110  setvbuf@plt
0x0000000000001120  fopen@plt
0x0000000000001130  sleep@plt
0x0000000000001140  _start
0x0000000000001170  deregister_tm_clones
0x00000000000011a0  register_tm_clones
0x00000000000011e0  __do_global_dtors_aux
0x0000000000001220  frame_dummy
0x0000000000001229  set
0x00000000000012de  main
0x00000000000013dc  _fini
(gdb)
```

2. Then “disas [address of main]. In my case it was 0x12de.

```
(gdb) disass 0x12de
Dump of assembler code for function main:
   0x00000000000012de <+0>:     endbr64
   0x00000000000012e2 <+4>:     push   %rbp
   0x00000000000012e3 <+5>:     mov    %rsp,%rbp
   0x00000000000012e6 <+8>:     sub    $0x10,%rsp
   0x00000000000012ea <+12>:    mov    $0x0,%eax
   0x00000000000012ef <+17>:    call   0x1229 <set>
   0x00000000000012f4 <+22>:    movl   $0x1,-0x4(%rbp)
   0x00000000000012fb <+29>:    lea    0xd1e(%rip),%rax        # 0x2020
   0x0000000000001302 <+36>:    mov    %rax,%rdi
   0x0000000000001305 <+39>:    call   0x10c0 <puts@plt>
   0x000000000000130a <+44>:    mov    $0x2,%edi
   0x000000000000130f <+49>:    call   0x1130 <sleep@plt>
   0x0000000000001314 <+54>:    lea    0xd1c(%rip),%rax        # 0x2037
   0x000000000000131b <+61>:    mov    %rax,%rdi
   0x000000000000131e <+64>:    call   0x10c0 <puts@plt>
   0x0000000000001323 <+69>:    mov    $0x2,%edi
   0x0000000000001328 <+74>:    call   0x1130 <sleep@plt>
   0x000000000000132d <+79>:    lea    0xd1a(%rip),%rax        # 0x204e
   0x0000000000001334 <+86>:    mov    %rax,%rdi
   0x0000000000001337 <+89>:    call   0x10c0 <puts@plt>
   0x000000000000133c <+94>:    mov    $0x2,%edi
   0x0000000000001341 <+99>:    call   0x1130 <sleep@plt>
   0x0000000000001346 <+104>:   lea    0xd18(%rip),%rax        # 0x2065
   0x000000000000134d <+111>:   mov    %rax,%rdi
   0x0000000000001350 <+114>:   call   0x10c0 <puts@plt>
   0x0000000000001355 <+119>:   mov    $0x2,%edi
   0x000000000000135a <+124>:   call   0x1130 <sleep@plt>
   0x000000000000135f <+129>:   mov    -0x4(%rbp),%eax
```

3.

```
0x00000000000012f4 <+22>:    movl   $0x1,-0x4(%rbp)
```

This line pushes the value 1 into ‘RBP’, which is the number used in the comparison check against 85. We can change it with the following commands (in this exact order):

b main

run

b *0x0000555555555302 (or any address after 0x1, -0x4(%rbp) instruction, but before the jle.

continue

set {int}($rbp-0x4) = 86 (any value over 85 will work)

continue

4.
```
(gdb) b *0x00005555555552fb
Breakpoint 2 at 0x5555555552fb
(gdb) continue
Continuing.

Breakpoint 2, 0x00005555555552fb in main ()
(gdb) set {int}($rbp-0x4) = 86
(gdb) continue
Continuing.
You roll for the loot.
Teammate 1 rolls a 40!
Teammate 2 rolls a 78!
Teammate 3 rolls a 85!
You roll a 86!
The loot is yours!
"flame{most_mythical_helmet}"
```

Flag acquired!!

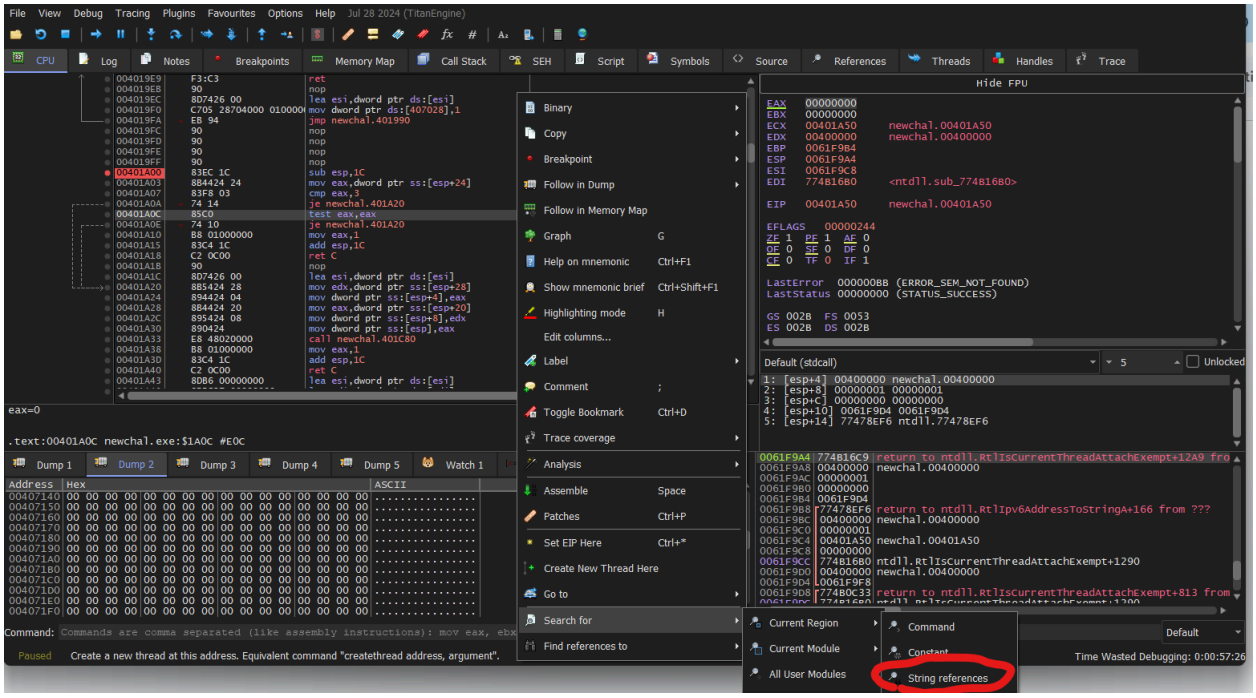
1. (x64DBG) When we first run the program, we can see that it runs, takes no input, and then closes.

```
C:\Users\ninth\Desktop\loot_

You roll for the loot.
Teammate 1 rolls a 40!
Teammate 2 rolls a 78!
Teammate 3 rolls a 85!
You roll a 1!
The loot goes to teammate 3!
```

2. Our first task is to locate main within the debugger.

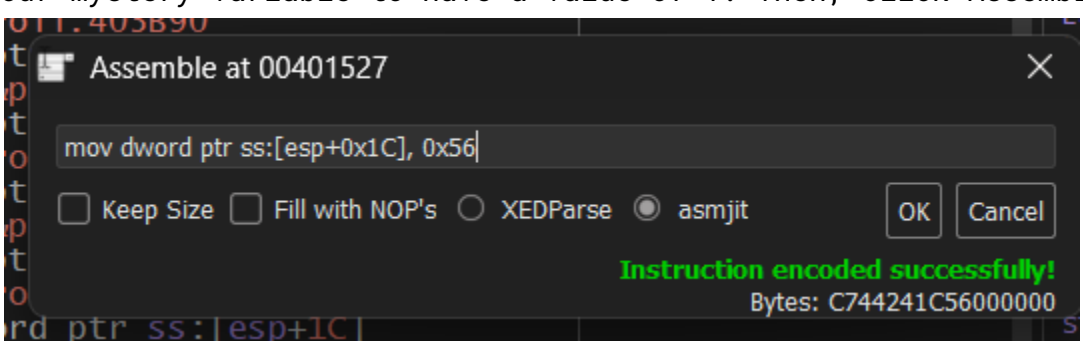
Right click on the CPU window, then Search For > All Modules > String References. (To open the code in x32dbg, go to File > Open).



3. Now, let's search for one of the strings. In my example, I search for "You roll a". Now, we have the assembly code of the entire function.

```
00401522   E8 09060000        call loot_roll.401B30
00401527   C74424 1C 01000000 mov dword ptr ss:[esp+1C],1
0040152F   C70424 80504000   mov dword ptr ss:[esp],loot_roll.405080  405080:"You roll for the loot."
00401536   E8 9D270000        call <JMP.&puts>
0040153B   C70424 02000000   mov dword ptr ss:[esp],2
00401542   E8 49260000        call loot_roll.403B90
00401547   C70424 97504000   mov dword ptr ss:[esp],loot_roll.405097  405097:"Teammate 1 rolls a 40!"
0040154E   E8 85270000        call <JMP.&puts>
00401553   C70424 02000000   mov dword ptr ss:[esp],2
0040155A   E8 31260000        call loot_roll.403B90
0040155F   C70424 AE504000   mov dword ptr ss:[esp],loot_roll.4050AE  4050AE:"Teammate 2 rolls a 78!"
00401566   E8 6D270000        call <JMP.&puts>
0040156B   C70424 02000000   mov dword ptr ss:[esp],2
00401572   E8 19260000        call loot_roll.403B90
00401577   C70424 C5504000   mov dword ptr ss:[esp],loot_roll.4050C5  4050C5:"Teammate 3 rolls a 85!"
0040157E   E8 55270000        call <JMP.&puts>
00401583   C70424 02000000   mov dword ptr ss:[esp],2
0040158A   E8 01260000        call loot_roll.403B90
0040158F   8B4424 1C         mov eax,dword ptr ss:[esp+1C]
00401593   894424 04         mov dword ptr ss:[esp+4],eax
00401597   C70424 DC504000   mov dword ptr ss:[esp],loot_roll.4050DC  4050DC:"You roll a %d!\n"
0040159E   E8 3D270000        call <JMP.&printf>
004015A3   837C24 1C 55      cmp dword ptr ss:[esp+1C],55             55:'U'
004015A8   7E 2F             jle loot_roll.4015D9
004015AA   C70424 EC504000   mov dword ptr ss:[esp],loot_roll.4050EC  4050EC:"The loot is yours!"
004015B1   E8 22270000        call <JMP.&puts>
004015B6   A1 70704000       mov eax,dword ptr ds:[407070]
004015BB   894424 04         mov dword ptr ss:[esp+4],eax
004015BF   C70424 FF504000   mov dword ptr ss:[esp],loot_roll.4050FF  4050FF:"%s"
004015C6   E8 15270000        call <JMP.&printf>
004015CB   C70424 0A000000   mov dword ptr ss:[esp],A                 0A:'\n'
004015D2   E8 B9250000        call loot_roll.403B90
004015D7   EB 0C             jmp loot_roll.4015E5
004015D9   C70424 02514000   mov dword ptr ss:[esp],loot_roll.405102  405102:"The loot goes to teammate 3!"
004015E0   E8 FB260000        call <JMP.&printf>
004015E5   C70424 0A000000   mov dword ptr ss:[esp],A                 0A:'\n'
004015EC   E8 9F250000        call loot_roll.403B90
004015F1   B8 00000000       mov eax,0
004015F6   C9                leave
004015F7   C3                ret
```

4. We see that there is a string that did not show up in our execution, “The loot is yours!”. This string is immediately preceded by a jle instruction, meaning that it is performing a conditional check. This conditional check is ss:[esp+1C] against 0x55. ss:[esp+1C] was loaded with the integer 1 (the second instruction in the screenshot), and 0x55 is 85 in decimal. So, if 1 is less than or equal to 85, the flag does not print.
5. Now, let’s make 1 > 85. Right click on the instruction that sets our mystery variable to have a value of 1. Then, click Assemble.



We now have the option to change the assembly code. Instead of 0x1, let’s change it to store 0x56.

Flag

```
flame{most_mythical_helmet}
```